Código de experimentación base con MLflow

Francisco Vicente Eugenio Ponce · A01840479
TC5061 · Actividad individual

Análisis de resultados

Para esta actividad se tomó la regresión logística del notebook de semana 3 y se pasó a train.py. Se conservaron las 23 variables y las reglas de limpieza: de 119,390 reservas quedaron 87,208. El preprocesamiento se ajustó solo con los datos de entrenamiento.

Se usaron 52,324 reservas para entrenamiento, 17,442 para validación y 17,442 para prueba, con semilla 42. En el experimento TC5061_Hoteles_Logistica_Experimentacion de MLflow se registraron siete configuraciones distintas y una repetición del baseline. Se comparó el efecto de C, class_weight y tol, manteniendo max_iter=1000 y un umbral de 0.5. La selección se hizo con F1 de validación.

La mejor configuración fue **04_balanced: C=1, class_weight=balanced, tol=0.0001 y max_iter=1000**. En validación obtuvo F1=0.5651, precisión=0.4698, recall=0.7090 y ROC-AUC=0.7696. Después de elegirla, el modelo guardado se evaluó en prueba sin volver a entrenarlo: F1=0.5665 y ROC-AUC=0.7770.

El parámetro que más influyó fue **class_weight**. Al pasar de none a balanced con los demás valores iguales, F1 subió de 0.4689 a 0.5651 y recall de 0.3499 a 0.7090. Esto permitió detectar más cancelaciones, pero la precisión bajó de 0.7105 a 0.4698: también hubo más falsas alarmas. ROC-AUC cambió muy poco, de 0.7691 a 0.7696, por lo que la mejora de F1 no implica que todas las métricas hayan mejorado. El balanceo cambia el peso de las clases en la función de pérdida durante el entrenamiento; no equivale a mover únicamente el umbral de decisión.

Cambiar C entre 0.01 y 100 tuvo un efecto menor: F1 estuvo entre 0.4587 y 0.4694 sin balanceo, y entre 0.5623 y 0.5651 con balanceo. Al usar tol=0.01, el entrenamiento pasó de 60 a 8 iteraciones; F1 subió a 0.4760, pero ROC-AUC bajó a 0.7627. Su F1 fue menor que el de la configuración elegida.

Las pequeñas diferencias entre valores de C no demuestran superioridad estadística. Las principales limitaciones son usar una sola partición y semilla, y no probar con reservas de periodos posteriores. Algunas variables, como booking_changes, podrían contener información conocida después de la reserva. Además, quitar duplicados puede eliminar reservas grupales legítimas. La partición de prueba ya se había usado en semana 3, aunque aquí no se utilizó para elegir el modelo. Antes de usarlo con un hotel sería necesario revisar estas decisiones y el costo de las falsas alarmas.

Reproducibilidad, ejecución y repositorio

Repositorio individual: https://github.com/FranciscoEP/TC5061-Experimentacion-MLFlow-A01840479

Se utilizó Python 3.13.5 y las versiones de requirements.txt. Se fijó semilla 42 en Python, NumPy, las particiones y el modelo, con un solo hilo de cálculo. La repetición produjo métricas y probabilidades idénticas (diferencia máxima=0), documentadas en results/summary.json. El SHA-256 del CSV se registra en MLflow para comprobar los datos utilizados.

Preparación y datos: seguir los pasos del README desde la raíz del proyecto, con el entorno virtual activo.

Entrenamiento y comparación:

python train.py

python train.py --C 1 --class-weight balanced --tol 0.0001 --max-iter 1000 --seed 42

python run_experiments.py

python finalize_results.py

python -m mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5001

El segundo comando usa la configuración elegida. Para reproducir el lote, ejecutar run_experiments.py y después finalize_results.py. Abrir MLflow en http://127.0.0.1:5001.

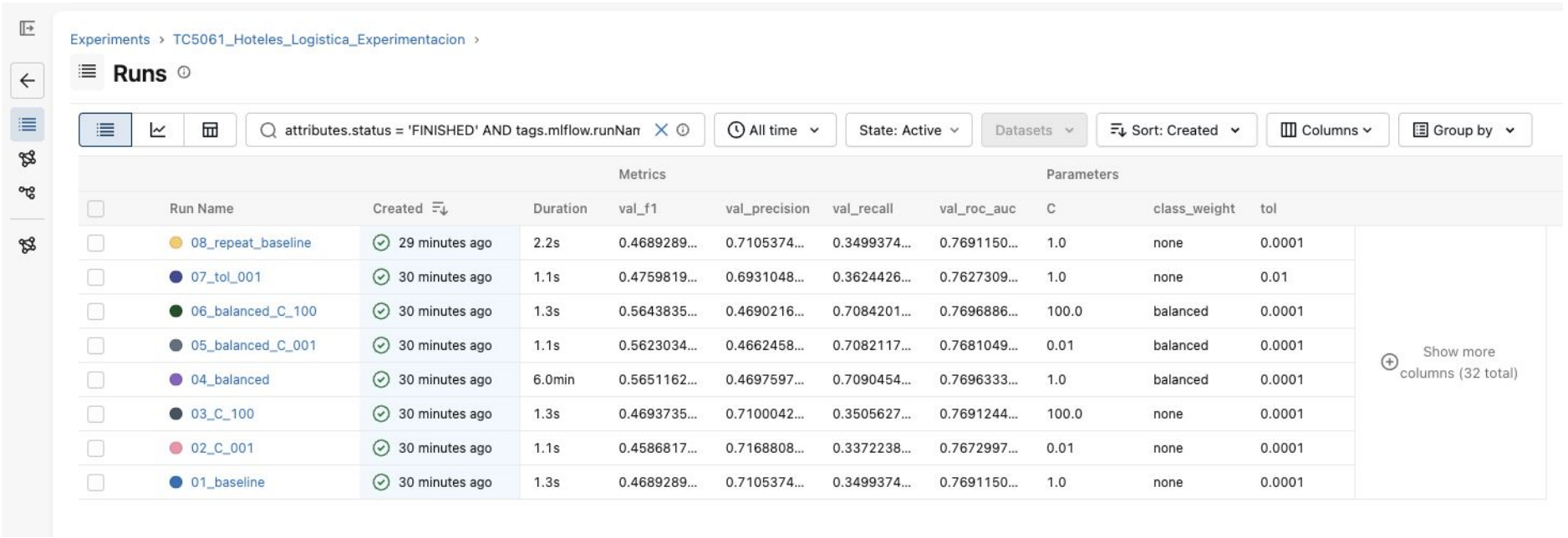
Tabla de validación (métricas redondeadas)

Corrida	C	Peso	tol	F1	Recall	AUC
01	1	none	0.0001	0.4689	0.3499	0.7691
02	0.01	none	0.0001	0.4587	0.3372	0.7673
03	100	none	0.0001	0.4694	0.3506	0.7691
04	1	balanced	0.0001	0.5651	0.7090	0.7696
05	0.01	balanced	0.0001	0.5623	0.7082	0.7681
06	100	balanced	0.0001	0.5644	0.7084	0.7697
07	1	none	0.01	0.4760	0.3624	0.7627
08	1	none	0.0001	0.4689	0.3499	0.7691

01: baseline; 02-03: C; 04-06: balanceo; 07: tolerancia; 08: repetición de 01. max_iter=1000 en todas. Las capturas de MLflow aparecen en los anexos.

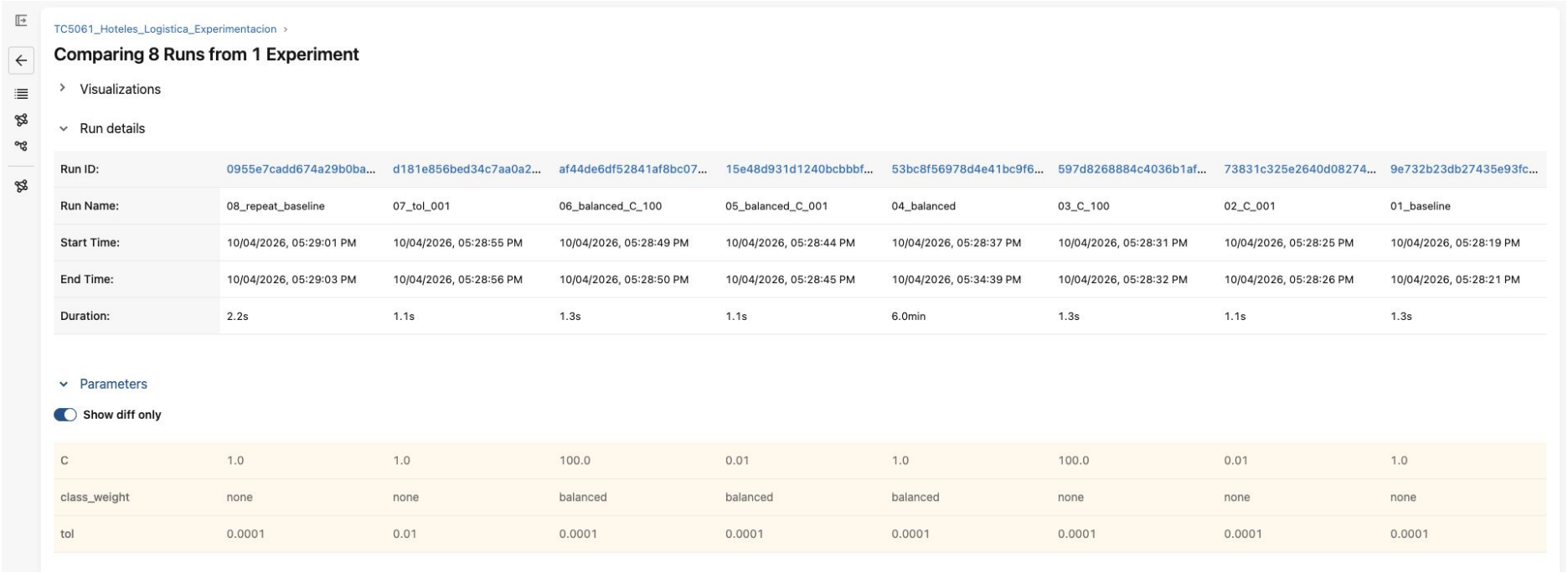
Fuentes: Hotel booking demand (Kaggle); MLflow sklearn; LogisticRegression (scikit-learn).

Anexo A. Ocho corridas: hiperparámetros y métricas de validación



Las ocho corridas pertenecen al mismo experimento. 04_balanced tiene el mayor F1; 01 y 08 repiten la misma configuración y resultado.

Anexo B. Vista Compare: detalles y parámetros de las ocho corridas



TC5061_Hoteles_Logistica_Experimentacion ›

Comparing 8 Runs from 1 Experiment

Visualizations

Run details

Run ID:	0955e7cadd674a29b0ba...	d181e856bed34c7aa0a2...	af44de6df52841af8bc07...	15e48d931d1240bcbbbf...	53bc8f56978d4e41bc9f6...	597d8268884c4036b1af...	73831c325e2640d08274...	9e732b23db27435e93fc...
Run Name:	08_repeat_baseline	07_tol_001	06_balanced_C_100	05_balanced_C_001	04_balanced	03_C_100	02_C_001	01_baseline
Start Time:	10/04/2026, 05:29:01 PM	10/04/2026, 05:28:55 PM	10/04/2026, 05:28:49 PM	10/04/2026, 05:28:44 PM	10/04/2026, 05:28:37 PM	10/04/2026, 05:28:31 PM	10/04/2026, 05:28:25 PM	10/04/2026, 05:28:19 PM
End Time:	10/04/2026, 05:29:03 PM	10/04/2026, 05:28:56 PM	10/04/2026, 05:28:50 PM	10/04/2026, 05:28:45 PM	10/04/2026, 05:34:39 PM	10/04/2026, 05:28:32 PM	10/04/2026, 05:28:26 PM	10/04/2026, 05:28:21 PM
Duration:	2.2s	1.1s	1.3s	1.1s	6.0min	1.3s	1.1s	1.3s

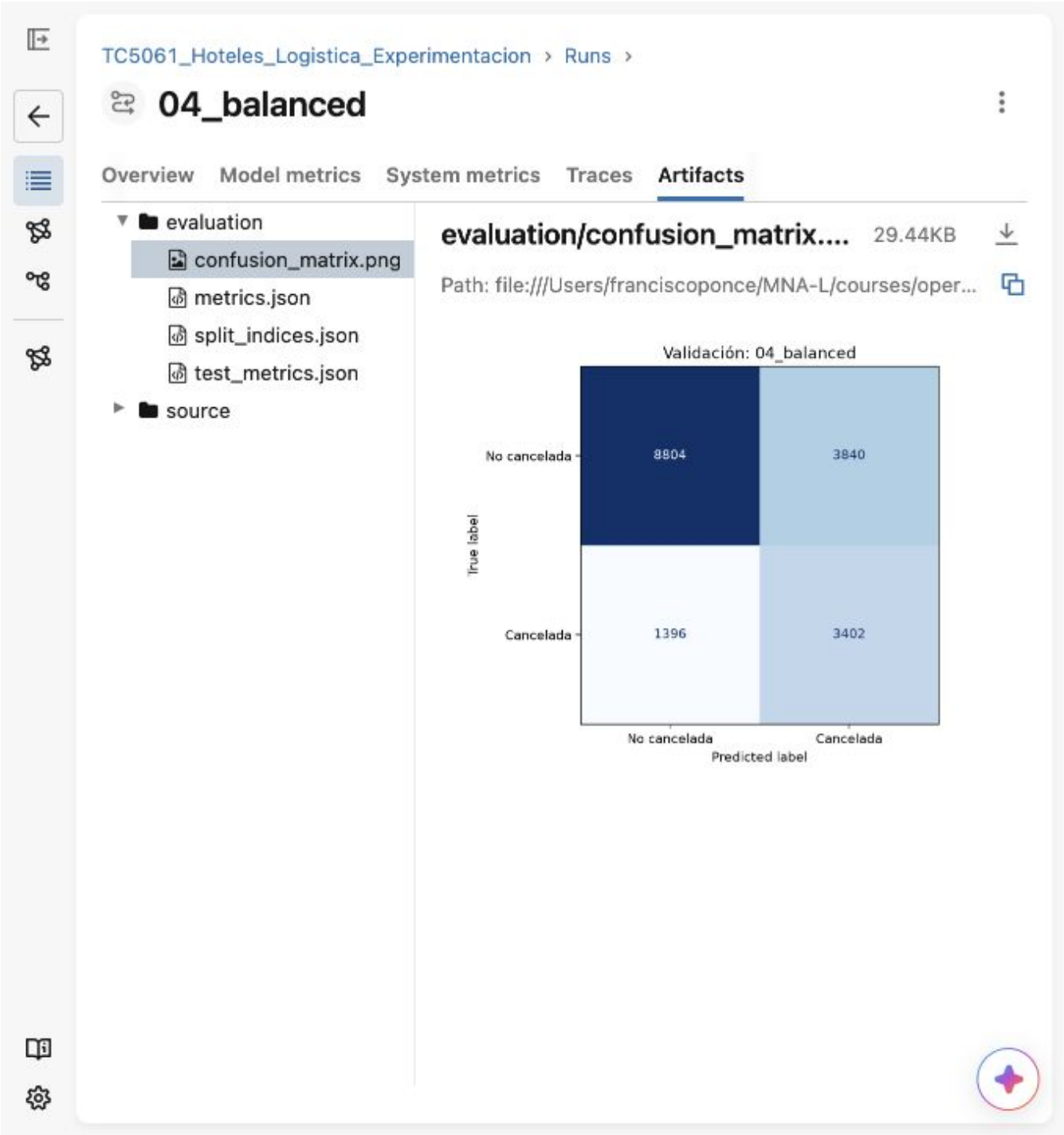
Parameters

Show diff only

C	1.0	1.0	100.0	0.01	1.0	100.0	0.01	1.0
class_weight	none	none	balanced	balanced	balanced	none	none	none
tol	0.0001	0.01	0.0001	0.0001	0.0001	0.0001	0.0001	0.0001

La vista Compare muestra los cambios de C, class_weight y tol. Las columnas corresponden a las corridas 08 a 01; sus métricas aparecen en el anexo A.

Anexo C. Detalle de 04_balanced y sus artefactos



La matriz de validación muestra 8,804 reservas no canceladas y 3,402 canceladas clasificadas correctamente, además de 3,840 falsas alarmas y 1,396 cancelaciones no detectadas. Se guardaron los artefactos y el pipeline completo con mlflow.sklearn.log_model.